Tarea 2

IMT2112 · Algoritmos Paralelos en Computación Científica · 2026-2

Viernes 2 de Octubre de 2026

- Trabajo en parejas.
- Los experimentos deben realizarse en al menos dos computadores distintos, y los resultados deben compararse entre sí.
- **Plazo de entrega:** Viernes 16 de octubre de 2026, 23:59.

Multiplicación de matrices con OpenMP y Numba

Multiplicar dos matrices densas $C = A \cdot B$ de tamaño $N \times N$ tiene un costo de $O(N^3)$ operaciones. Sobre este problema estudiaremos tres versiones paralelas: dos en C++ con OpenMP y una en Python con Numba.

Algoritmo 1: Paralelismo de datos sobre el problema base

La forma directa de calcular el producto son tres *loops* anidados:

```
for (int i = 0; i < N; ++i) {
    for (int k = 0; k < N; ++k) {
        for (int j = 0; j < N; ++j) {
            C[i*N + j] += A[i*N + k] * B[k*N + j];
        }
    }
}
```

Sobre este cómputo se aplicará paralelismo de datos.

Algoritmo 2: Dividir y Conquistar

Una forma alternativa es dividir y conquistar: se parte cada matriz en cuatro cuadrantes y el producto se expresa en productos de bloques más pequeños,

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \qquad C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}.$$

Cada cuadrante se subdivide de nuevo, hasta llegar a bloques de tamaño $n \times n$ (las hojas), donde se realiza una multiplicación densa directa que cuesta $O(n^3)$. Esto define un árbol de bloques.

El tamaño de bloque n controla la forma del árbol:

- la profundidad es $\log_2(N/n)$ y hay $(N/n)^2$ hojas por matriz;
- n pequeño $\Rightarrow$ árbol profundo, con muchas hojas baratas;
- n grande $\Rightarrow$ árbol plano, con pocas hojas costosas.

Así se puede estudiar el efecto de la granularidad, la memoria caché y el *overhead*.
La estructura de árbol expone un paralelismo distinto al del Algoritmo 1: los subproductos de distintos cuadrantes son independientes, lo que permite aplicar paralelismo de tareas.

Algoritmo 3: Prototipado rápido en Python con Numba

Numba[1] es un compilador JIT que traduce funciones de Python a código nativo. Con el decorador `@njit(parallel=True)` y el *loop* `prange` paraleliza automáticamente los *loops* de datos.[2]

Su ventaja es el prototipado rápido: muy buen nivel de paralelismo con muy poco código. A cambio, ofrece poco control sobre cómo se paraleliza: su modelo es solo de *loops* de datos, sin paralelismo de tareas ni control fino del *scheduling* o la afinidad de *threads*. Es la herramienta natural para el Algoritmo 1 en Python.

[1] Documentación: https://numba.readthedocs.io
[2] Paralelización automática: https://numba.readthedocs.io/en/stable/user/parallel.html

Código provisto

Se entrega `serial.cpp`, que permite ejecutar las versiones secuenciales de los Algoritmos 1 y 2. Las versiones paralelas deben implementarse a partir de él, verificando en cada una que el resultado coincide con la referencia. Use $N = 2048$ salvo que se indique otra cosa. Para que los tiempos sean comparables entre versiones y computadores, compile el C++ con `-O3 -march=native -fopenmp` o equivalentes.

(a) **[15 pts]** *Base serial y tamaño de bloque.* Compile y ejecute `serial.cpp`. Para $N = 2048$, mida el tiempo variando el tamaño de bloque $n \in \{2048, 1024, 512, 256, 128, 64, 32\}$ (note que $n = N$ corresponde al problema base, sin subdivisión). Presente los tiempos en una tabla o gráfico. Comente sus resultados.

(b) **[15 pts]** *Algoritmo 1: paralelismo de datos.* Implemente en C++ la versión de paralelismo de datos sobre la multiplicación densa $N \times N$ usando OpenMP. Mida el tiempo con $p \in \{1, 2, 4, 8\}$ *threads* y calcule el *speedup* $S(p) = T(1)/T(p)$ y la eficiencia $E(p) = S(p)/p$.

(c) **[20 pts]** *Algoritmo 2: dividir y conquistar.* A partir de `serial.cpp`, implemente la versión paralela de dividir y conquistar usando OpenMP. Comente los detalles de su implementación. Justifique la elección de un valor de n y mida el tiempo con $p \in \{1, 2, 4, 8\}$ *threads*, calculando el *speedup* $S(p) = T(1)/T(p)$ y la eficiencia $E(p) = S(p)/p$.

(d) **[15 pts]** *Algoritmo 3: Numba en Python.* Implemente desde cero, en Python, la versión de paralelismo de datos (Algoritmo 1) usando Numba. Mida el tiempo con $p \in \{1, 2, 4, 8\}$ *threads* y compárelo con las otras versiones; recuerde excluir el tiempo de compilación JIT con una llamada de *warm-up* antes de medir.

(e) **[10 pts]** *Comparación y escalamiento.* Grafique $T(p)$, $S(p)$ y $E(p)$ en función de p para las tres versiones, comparando con la curva ideal. Justifique su elección de $T(1)$. Para el Algoritmo 2, muestre cómo cambia el resultado al variar el tamaño de bloque n. Comente sus resultados.

(f) **[15 pts]** *Escalabilidad débil.* Repita las mediciones para $N \in \{1024, 2048, 4096\}$ y estudie cómo cambia la eficiencia débil a medida que crece el tamaño del problema. ¿Se mantiene el desempeño paralelo al aumentar N? Presente sus resultados en una tabla o gráfico y comentélos.

(g) **[10 pts]** *Segundo computador.* Repita los experimentos en un segundo computador. Para cada uno, declare el número de cores físicos y lógicos y la velocidad de reloj. Compare los resultados y justifique las diferencias.

Instrucciones de entrega:

- Entregar un documento PDF con sus resultados a través de Canvas.
- El uso de IA está permitido, mientras esté debidamente declarado en el informe.
- Se evaluará la claridad y legibilidad de los gráficos y tablas. Es importante identificar títulos, *labels*, leyendas y tamaño de fuente adecuados.